

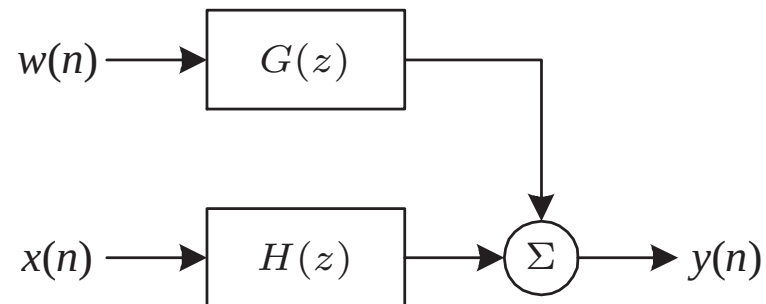
Joint Signal Analysis Overview

- Cross-correlation
- Cross Power Spectrum and transfer function estimation
- Example

Introduction

- Mostly we have focused on estimating statistical properties of a single *univariate* signal
 - Autocorrelation function (ACF)
 - Partial autocorrelation function (PACF)
 - Power spectral density
- In many applications we have two or more signals, $x(n)$ and $y(n)$
- Would like to say something about how they are related

System Identification

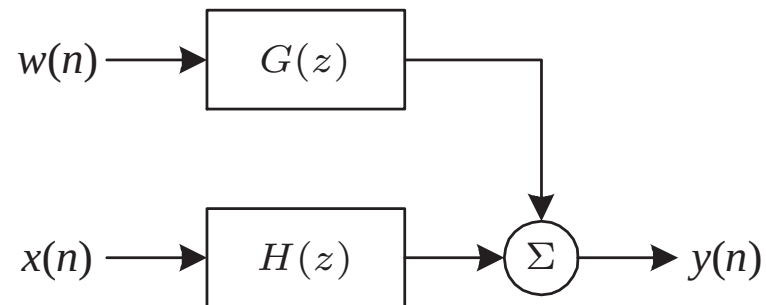


- Joint signal analysis is related to system identification
- The goal of system identification is to build a model
- That is, estimate $H(z)$ and $G(z)$, given $x(n)$ and $y(n)$
 - Parametric, though order may be estimated
 - Mostly LTI systems
 - Some methods for MIMO systems

Joint Signal Analysis

- Joint signal analysis characterizes the relationship between a pair of signals
 - We will focus on nonparametric methods
 - LTI systems
 - Only 2 signals
- We have already discussed many of the possible properties
 - Normalized cross-correlation, aka cross-correlation function (CCF)
 - Cross-power spectral density (CPSD)
 - Coherence

Joint Signal Analysis



- If we assume that $x(n)$ and $y(n)$ are related by an LTI system $H(z)$, then we can estimate the properties of $H(z)$ given realizations of $x(n)$ and $y(n)$
 - Magnitude
 - Phase
 - Group delay
 - Impulse response

Causality

- Strictly speaking, given only realizations of $x(n)$ and $y(n)$ we cannot determine whether one signal *caused* the other or not
- This is a fundamental idea rooted in statistics
- However, under certain assumptions causality can be determined
- For example, if we assume that any system that relates $x(n)$ to $y(n)$ or vice versa is causal, then we may be able to get a sense of which caused which from the estimated cross-correlation

Normalized Cross-Correlation Defined

Normalized Cross-Correlation, also known as the **Cross-correlation Function (CCF)** is defined for a WSS signal as

$$\rho_{yx}(\ell) = \frac{\gamma_{yx}(\ell)}{\sqrt{\gamma_y(0)\gamma_x(0)}} = \frac{\gamma_{yx}(\ell)}{\sigma_y\sigma_x}$$

where $\gamma_{yx}(\ell)$ is the cross-covariance of $y(n)$ and $x(n)$,

$$\gamma_{yx}(\ell) = \text{E}[(y(n + \ell) - \mu_y)(x(n) - \mu_x)^*]$$

Estimated Cross-Covariance

$$\hat{\gamma}_u(\ell) \triangleq \frac{1}{N - |\ell|} \sum_{n=0}^{N-1-|\ell|} \begin{cases} [y(n + \ell) - \hat{\mu}_y] [x(n) - \hat{\mu}_x]^* & 0 \leq \ell \leq N - 1 \\ [y(n) - \hat{\mu}_y] [x(n - \ell) - \hat{\mu}_x]^* & -(N - 1) \leq \ell \leq 0 \\ 0 & \text{Otherwise} \end{cases}$$

$$\hat{\gamma}_b(\ell) \triangleq \frac{N - |\ell|}{N} \hat{\gamma}_u(\ell)$$

- There are similar trade-offs between biased and unbiased estimates of the CCF as there were for the ACF
- As with ACF, we generally assume the means are zero to simplify calculations and eliminate DC artifacts
- The bias caused by estimating the means is usually negligible

Estimated Cross-Covariance: Biased versus Unbiased

If $\mu_y = \mu_x = 0$,

$$\hat{\gamma}_u(\ell) \triangleq \frac{1}{N - |\ell|} \sum_{n=0}^{N-1-|\ell|} \begin{cases} y(n + \ell)x(n)^* & 0 \leq \ell \leq N - 1 \\ y(n)x(n - \ell)^* & -(N - 1) \leq \ell \leq 0 \\ 0 & \text{Otherwise} \end{cases}$$

$$\hat{\gamma}_b(\ell) \triangleq \frac{N - |\ell|}{N} \hat{\gamma}_u(\ell)$$

- The true CCF is not positive definite and the cross-spectral density is not non-negative, in general, so this is no longer a concern
- Nonetheless, the unbiased estimator contains excessive variance at large lags and thus the biased estimate is generally preferred

Estimated Cross-Correlation Functions

The natural estimators of the CCF are

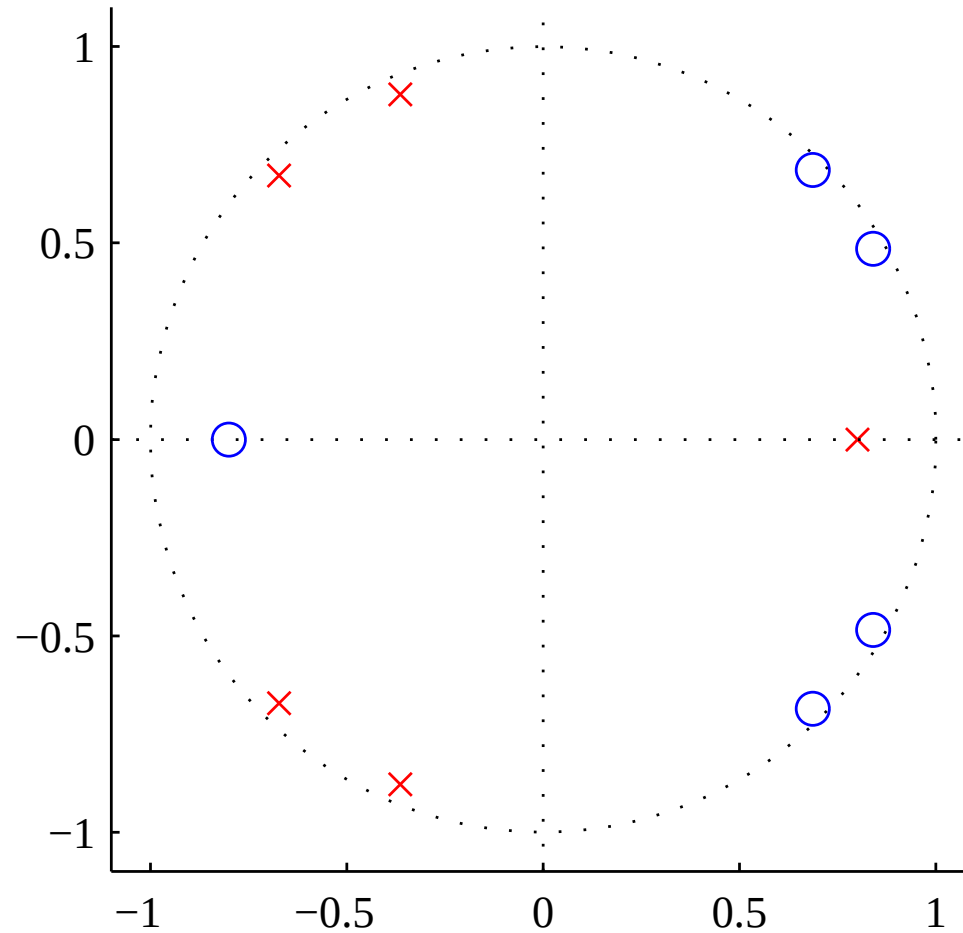
$$\hat{\rho}_b(\ell) \triangleq \frac{\hat{\gamma}_b(\ell)}{\hat{\sigma}_x \hat{\sigma}_y} \qquad \hat{\rho}_u(\ell) \triangleq \frac{\hat{\gamma}_u(\ell)}{\hat{\sigma}_x \hat{\sigma}_y}$$

- In general, it is not possible to obtain confidence intervals for the estimated CCF because the variance of the estimator depends on the true CCF and signal ACF's
- Instead, it is common practice to plot the confidence intervals of a purely random process
- If N is large enough, the central limit theorem applies and $\text{var}\{\hat{\rho}_b(\ell)\}$ is approximately normal for all ℓ
- In this case, we can use the Normal CDF to plot confidence intervals of an IID sequence
- These are proportional to $\pm \sqrt{\text{var}\{\hat{\rho}_{yx}(\ell)\}}$
- This is the same as it was for the ACF

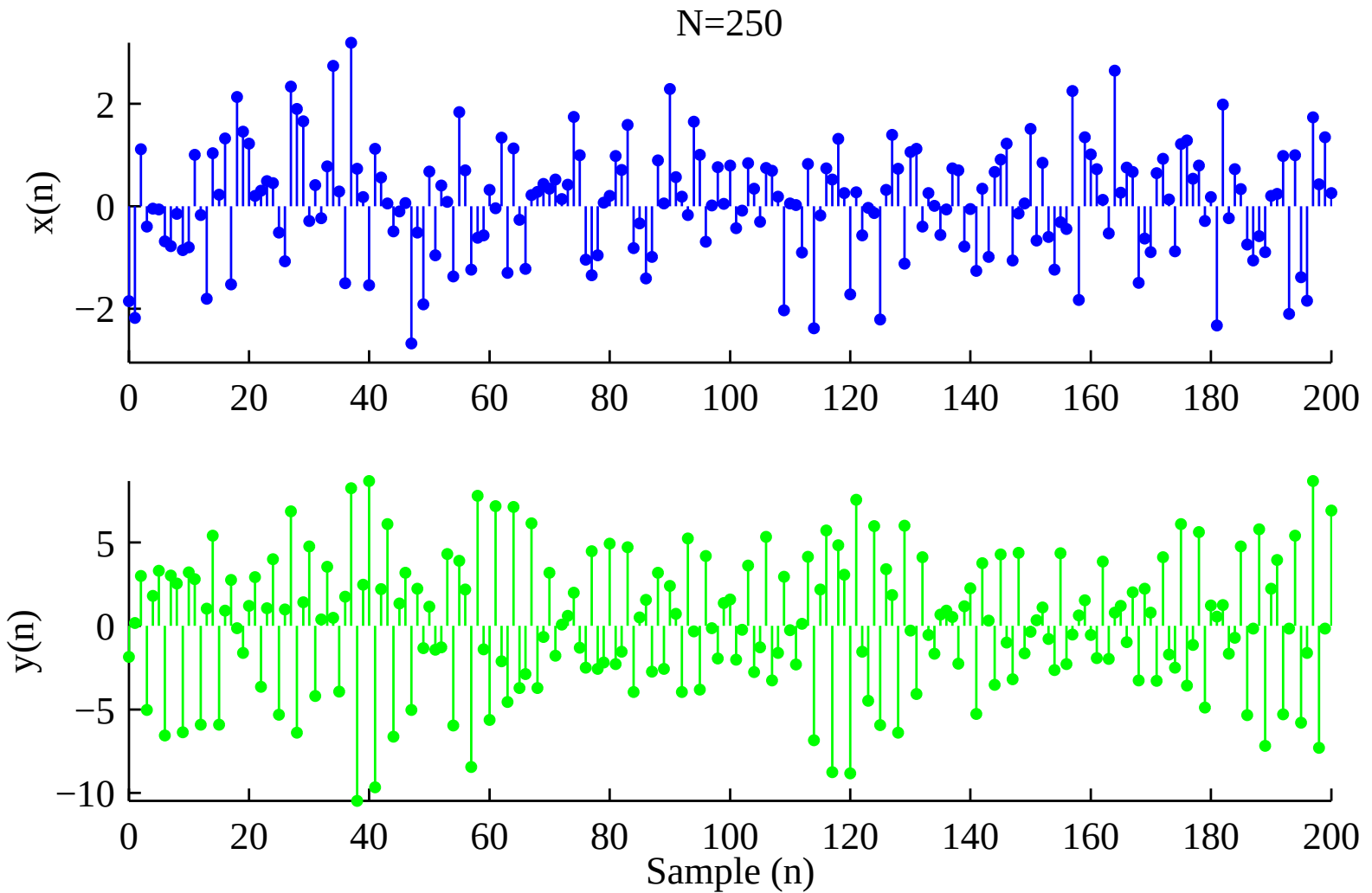
Example 1: CCF Estimation

Estimate the CCF of the ARMA process investigated earlier for PSD estimation.

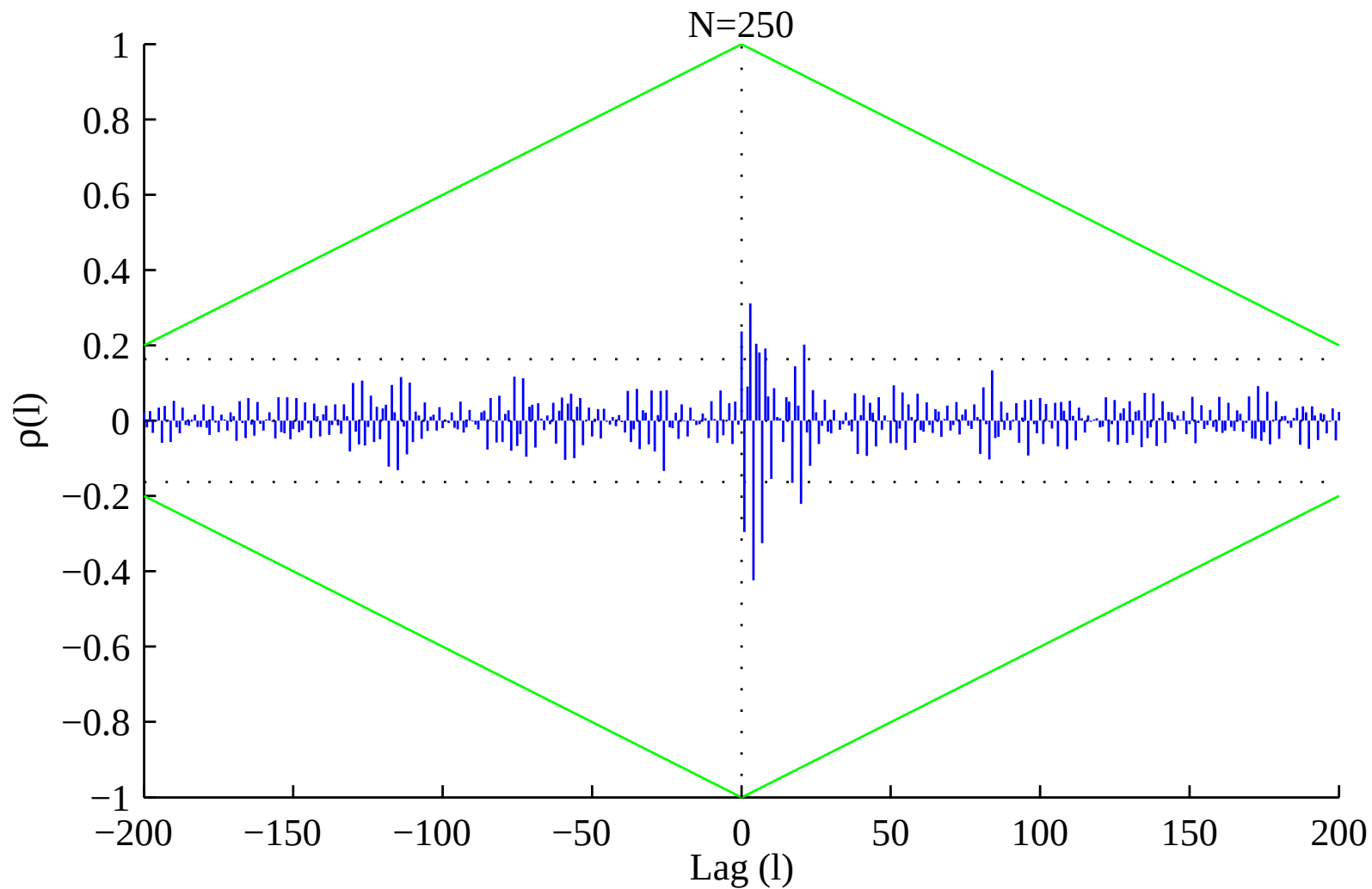
Example 1: Pole Zero Map of ARMA Process



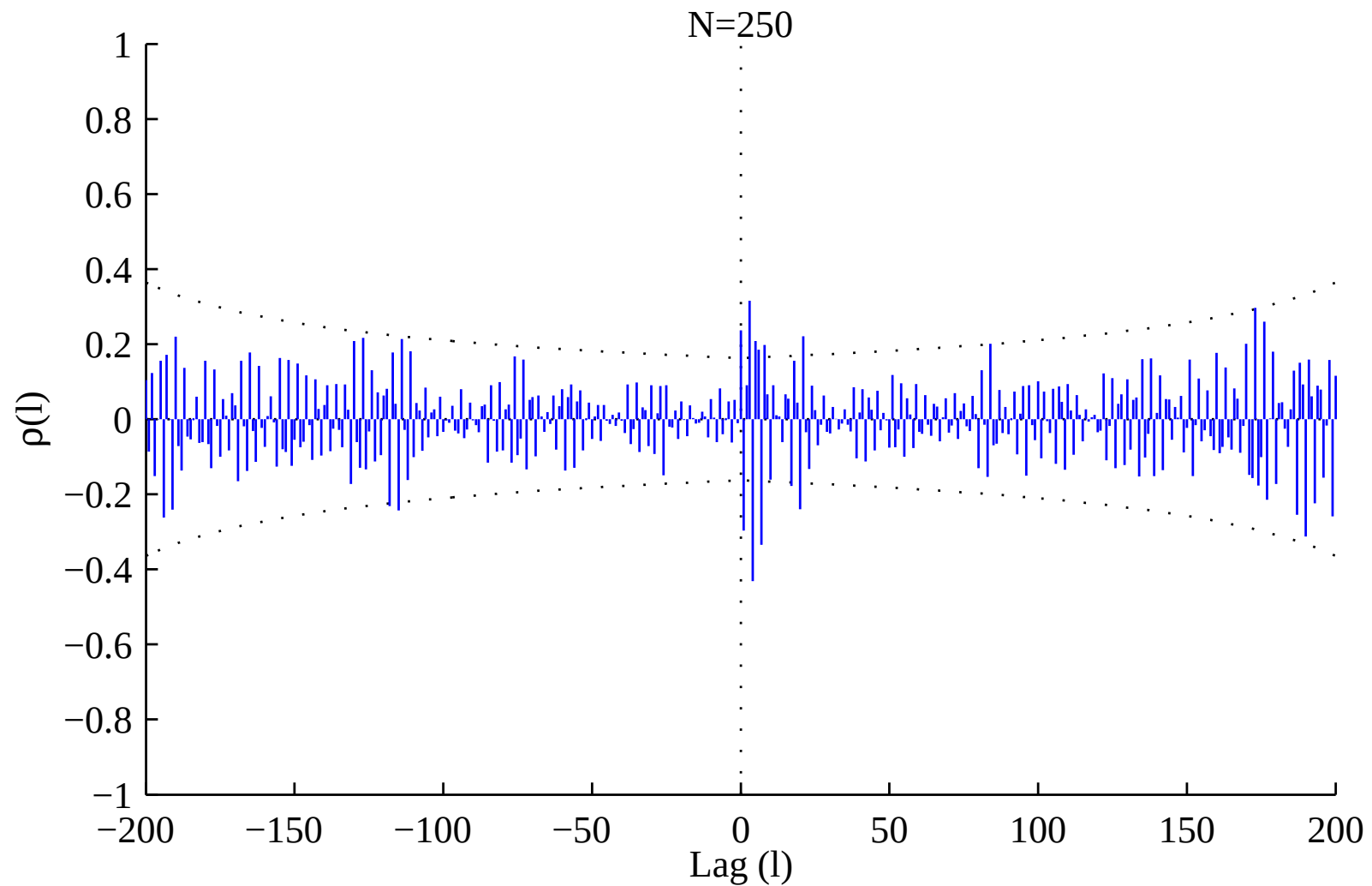
Example 1: Signals



Example 1: Biased CCF



Example 1: Unbiased CCF



Example 1: MATLAB Code

```
%function [] = CrossCorrelationEx();
close all;
clear all;

L = 200; % Length of autocorrelation calculated
N = 250;
c1 = 99; % Confidence level
NP = norminv((1-c1/100)/2); % Find corresponding lower percentile of Normal

%=====
% Create Signals
%=====
b = poly([-0.8,0.97*exp(j *pi/4),0.97*exp(-j *pi/4),...
          0.97*exp(j *pi/6),0.97*exp(-j *pi/6)]); % Numerator
a = poly([ 0.8,0.95*exp(j*3*pi/4),0.95*exp(-j*3*pi/4),...
          0.95*exp(j*2.5*pi/4),0.95*exp(-j*2.5*pi/4)]); % Denominator
b = b*sum(a)/sum(b); % Set DC gain to 1

x = randn(N,1);
y = filter(1,a,x);

%=====
% Plot the Pole-Zero Map
%=====
figure
FigureSet(1,2.8,2.8);
h = circle;
```

```

z = roots(b);
p = roots(a);
hold on;
    h2 = plot(real(z),imag(z),'bo',real(p),imag(p),'rx');
    hold off;
axis square;
xlim([-1.1 1.1]);
ylim([-1.1 1.1]);
axislines;
box off;
AxisSet(8);
print -depsc SystemPoleZeroMap;

%=====
% Plot the Signals
%=====

figure
FigureSet(1,'LTX');
n = 0:min(200,N);
subplot(2,1,1);
    h = stem(n,x(n+1));
    set(h(1),'MarkerFaceColor','b');
    set(h(1),'MarkerSize',2);
    xlim([0 max(n)]);
    ylim([min(x) max(x)]);
    ylabel('x(n)');
    title(sprintf('N=%d',N));
    box off;
subplot(2,1,2);
    h = stem(n,y(n+1),'g');
    set(h(1),'MarkerFaceColor','g');

```

```

        set(h(1), 'MarkerSize', 2);
        xlim([0 max(n)]);
        ylim([min(y) max(y)]);
        ylabel('y(n)');
        xlabel('Sample (n)');
        box off;
AxisSet(8);
print -depsc JointSignals;

%=====
% Plot the Biased CCF
%=====
np = 2^nextpow2(2*N-1);    % Figure out how much to pad the signal
Y = fft(y,np);
X = fft(x,np);
cp = ifft(Y.*conj(X));
cc = real([cp(np-(L-1:-1:0)); cp(1:L+1)]);
cc = cc/(std(y)*std(x));
cc = cc/N;
l = (-L:L).';

figure
FigureSet(1, 'LTX');
h = stem(l, cc);
set(h(1), 'Marker', 'None');
xlim([-L L]);
ylim([-1 1]);
hold on;
plot(xlim, NP*[1 1]/sqrt(N), 'k:', xlim, -NP*[1 1]/sqrt(N), 'k:');
plot([ 0 L], [1 (N-L)/N], 'g', [ 0 L], -[1 (N-L)/N], 'g');
plot([-L 0], [(N-L)/N 1], 'g', [-L 0], -[(N-L)/N 1], 'g');

```

```

        hold off;
ylabel('\rho(l)');
xlabel('Lag (l)');
title(sprintf('N=%d',N));
box off;
AxisLines;
AxisSet(8);
print -depsc CrossCorrelationBiased;

%=====
% Plot the Unbiased ACF
%=====
np = 2^nextpow2(2*N-1);    % Figure out how much to pad the signal
Y = fft(y,np);
X = fft(x,np);
cp = ifft(Y.*conj(X));
cc = real([cp(np-(L-1:-1:0));cp(1:L+1)]);
cc = cc/(std(y)*std(x));
l = (-L:L).';
cc = cc./(N-abs(l));

figure
FigureSet(1,'LTX');
h = stem(l,cc);
set(h(1),'Marker','None');
xlim([-L L]);
ylim([-1 1]);
hold on;
    plot(l,NP*1./sqrt(N-abs(l)),'k:',l,-NP*1./sqrt(N-abs(l)),'k:');
    hold off;
ylabel('\rho(l)');

```

```
xlabel('Lag (1)');  
title(sprintf('N=%d',N));  
box off;  
AxisLines;  
AxisSet(8);  
print -depsc CrossCorrelationUnbiased;
```

Cross-Power Spectral Density

Recall that the cross-power spectral density (CPSD) is defined as

$$R_{yx}(e^{j\omega}) \triangleq \sum_{\ell=-\infty}^{\infty} r_{yx}(\ell)e^{-j\omega\ell}$$

- Unlike the PSD, the CPSD is a complex-valued function
- Can work with CPSD in either rectangular or polar form
- Rectangular
 - **Cospectrum:** $C_{yx}(\omega) \triangleq \text{Re}\{R_{yx}(e^{j\omega})\}$
 - **Quadrature spectrum:** $Q_{yx}(\omega) \triangleq \text{Im}\{R_{yx}(e^{j\omega})\}$
- More commonly, it is expressed in terms of magnitude and phase
 - **Cross-Amplitude Spectrum:** $A_{yx}(\omega) \triangleq |R_{yx}(e^{j\omega})|$
 - **Phase Spectrum:** $\Phi_{yx}(\omega) \triangleq \angle R_{yx}(e^{j\omega})$

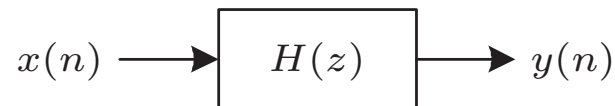
Estimating Cross-Power Spectral Density

The **Cross-periodogram** is defined as

$$\begin{aligned}\hat{R}_{yx}(e^{j\omega}) &\triangleq \sum_{\ell=-(N-1)}^{N-1} \hat{r}_{yx}(\ell) e^{-j\omega\ell} \\ &= \frac{1}{N} \left[\sum_{n=0}^{N-1} y(n) e^{-j\omega n} \right] \left[\sum_{n=0}^{N-1} x(n) e^{-j\omega n} \right]^*\end{aligned}$$

- We may use the same nonparametric techniques to estimate the CPSD that we used to estimate the PSD
- The same tradeoffs exist
 - Main lobe width versus side lobe height
 - Bias versus variance
- Can reduce variance at the expense of bias using either smoothing (Blackman-Tukey) or averaging (Welch-Bartlett)

Estimating Frequency Response



$$y(n) = h(n) * x(n)$$

$$r_{yx}(\ell) = h(\ell) * r_x(\ell)$$

$$R_{yx}(e^{j\omega}) = H(e^{j\omega})R_x(e^{j\omega})$$

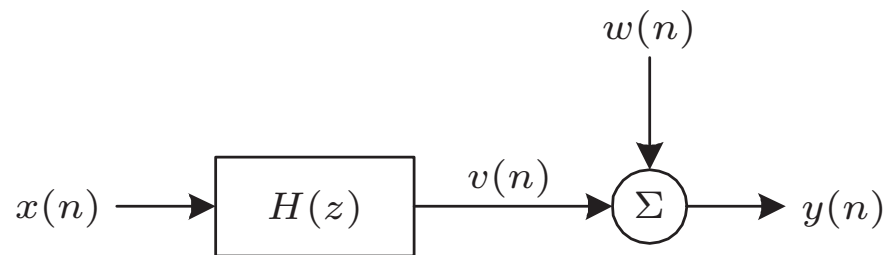
$$H(e^{j\omega}) = \frac{R_{yx}(e^{j\omega})}{R_x(e^{j\omega})}$$

- If $x(n)$ and $y(n)$ are the input and output of a system, we can use these techniques to estimate properties of the system $H(z)$
- Unlike the univariate case, if we know both $x(n)$ and $y(n)$ there is no ambiguity between minimum-phase and non-minimum phase systems
- Note that although we may be able to estimate $H(e^{j\omega})$, this does not mean we have a parametric description of it

Which CPSD Estimate?

- The book argues that Welch's approach to modified periodogram averaging is preferred
- Not clear to me why they prefer this to Blackman-Tukey, which I think is better

Additive Noise

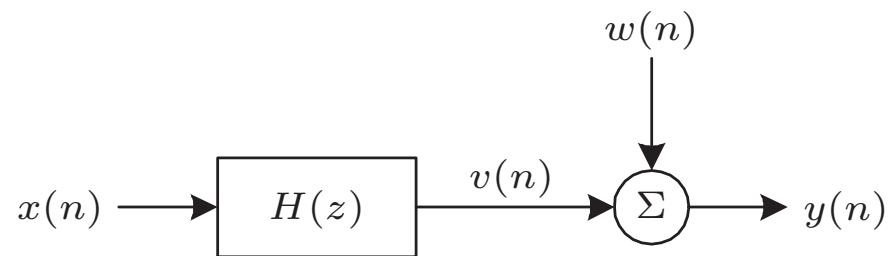


$$R_{yx}(e^{j\omega}) = R_{vx}(e^{j\omega}) \qquad H(e^{j\omega}) = \frac{R_{vx}(e^{j\omega})}{R_x(e^{j\omega})} = \frac{R_{yx}(e^{j\omega})}{R_x(e^{j\omega})}$$

$$R_y(e^{j\omega}) = R_v(e^{j\omega}) + R_w(e^{j\omega}) \qquad R_v(e^{j\omega}) = |H(e^{j\omega})|^2 R_x(e^{j\omega})$$

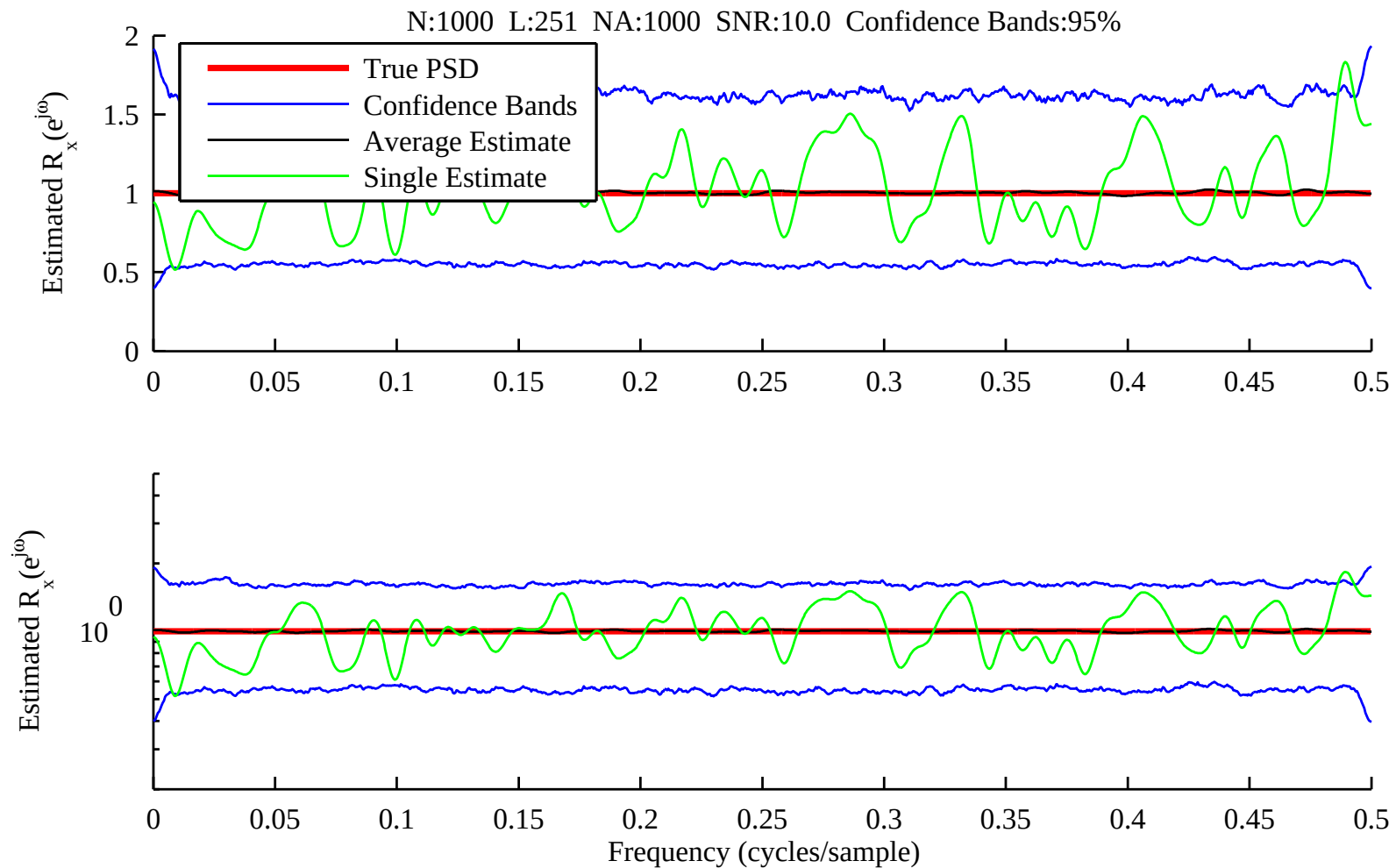
- It is instructive to consider the case of additive noise
- Here we assume that $w(n)$ is statistically independent of $x(n)$ and all signals are stationary with zero mean
- Note: different notation than text

Example 2: Blackman-Tukey Estimate

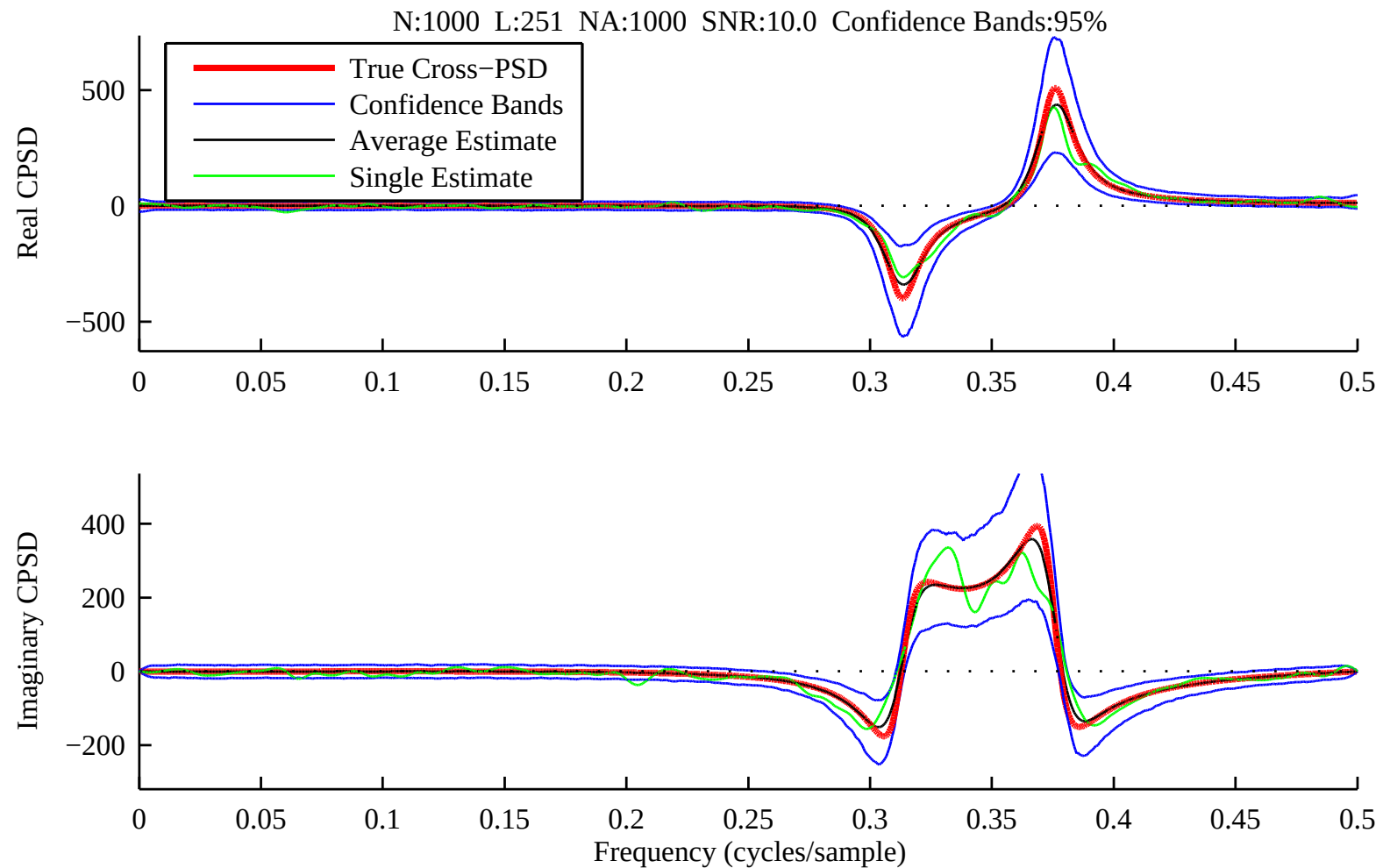


Use the Blackman-Tukey estimate to estimate the PSDs of $x(n)$ and $y(n)$ for various signal to noise ratios (SNRs). Also estimate the transfer function magnitude and phase for the PZ system described earlier.

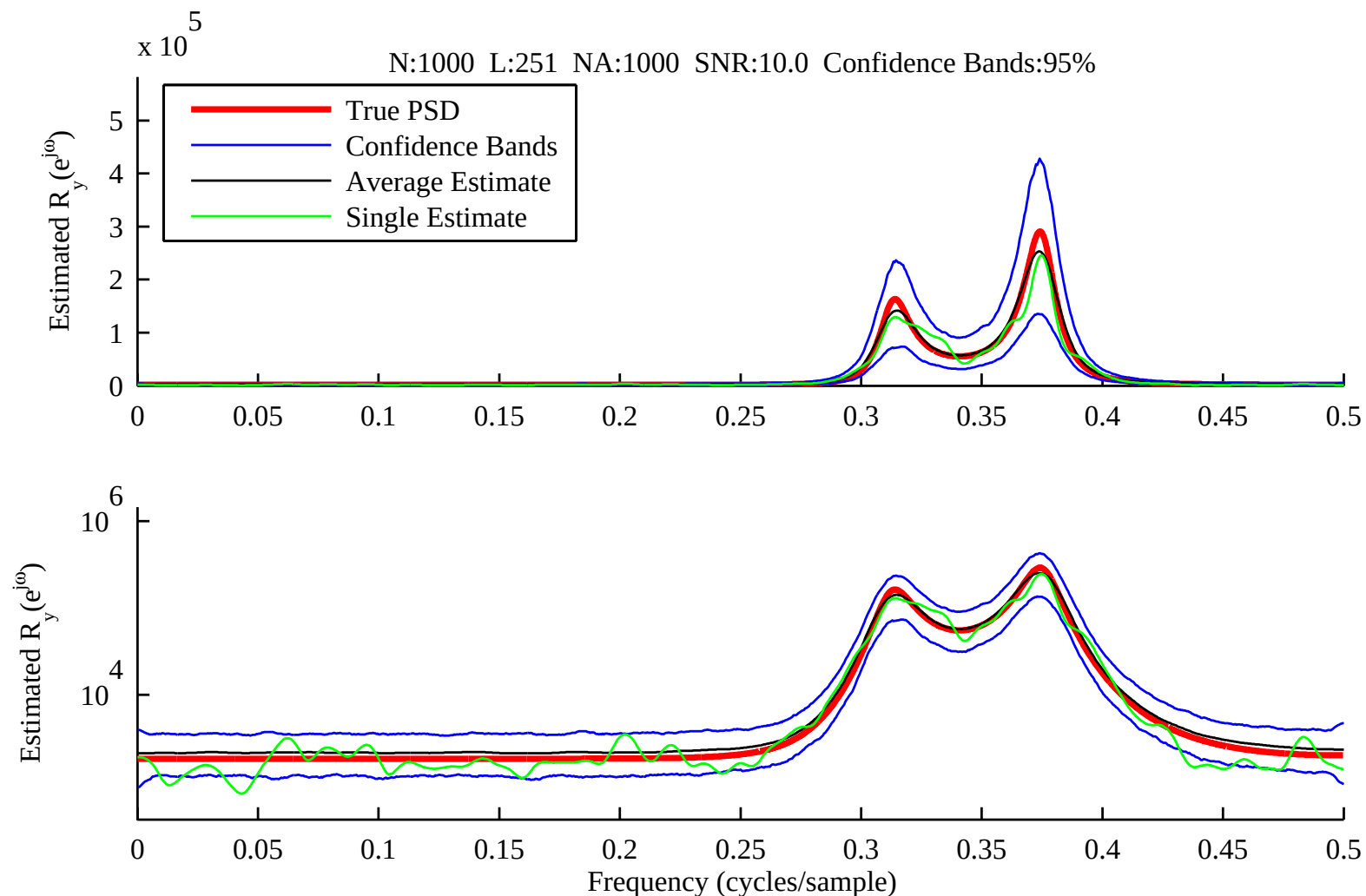
Example 2: Blackman-Tukey Estimate $R_x(e^{j\omega})$



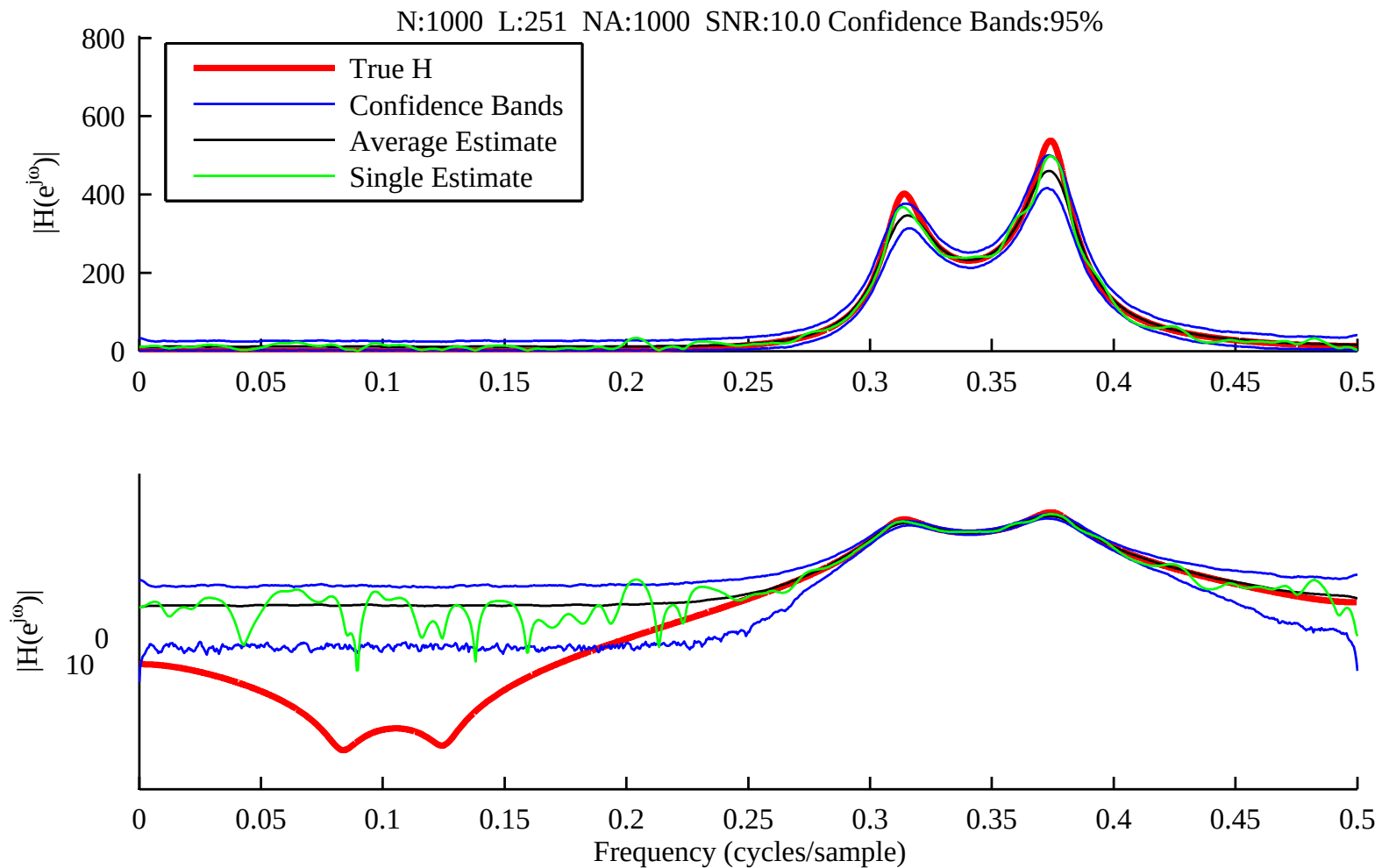
Example 2: Blackman-Tukey Estimate $R_{yx}(e^{j\omega})$



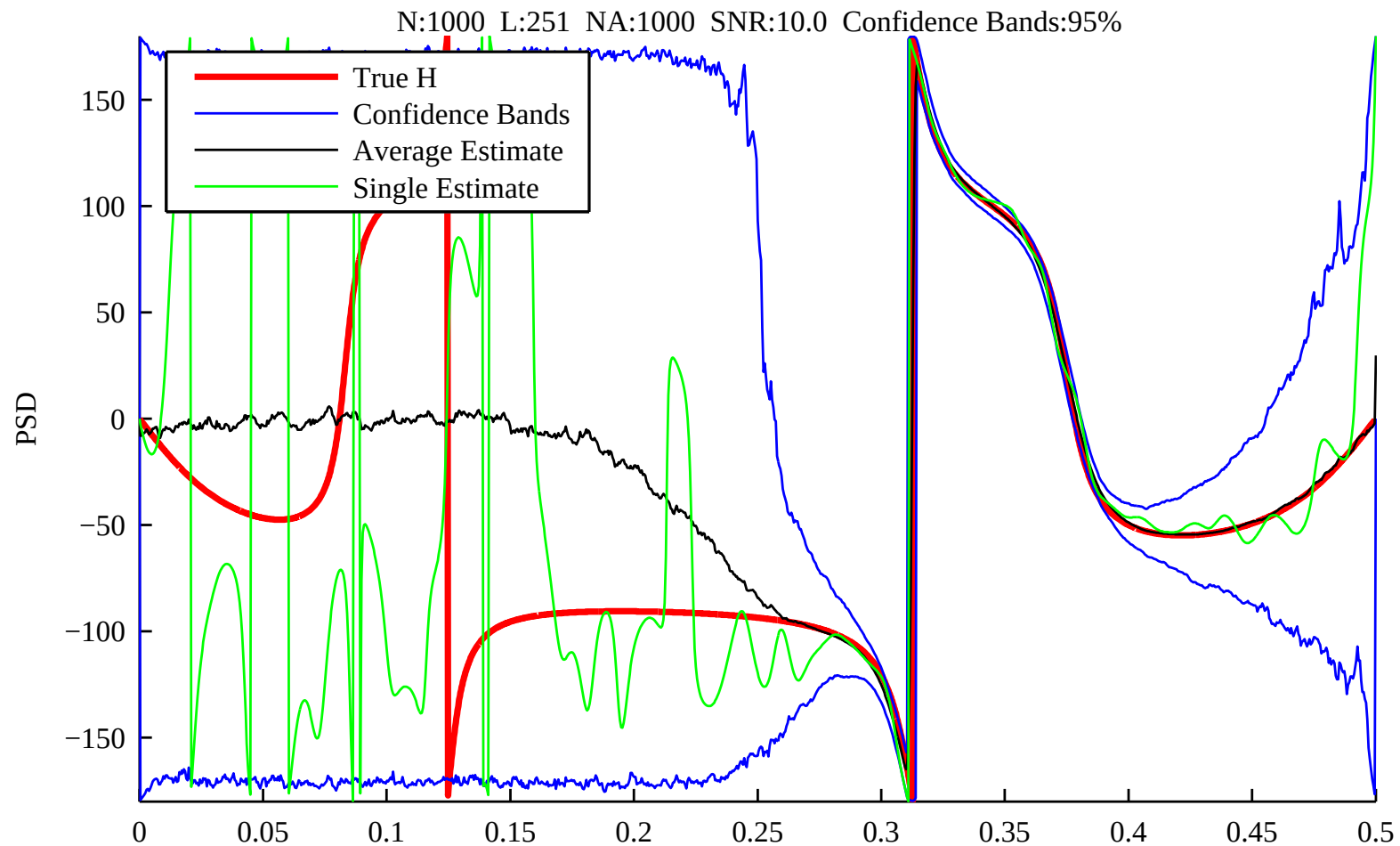
Example 2: Blackman-Tukey Estimate $R_y(e^{j\omega})$



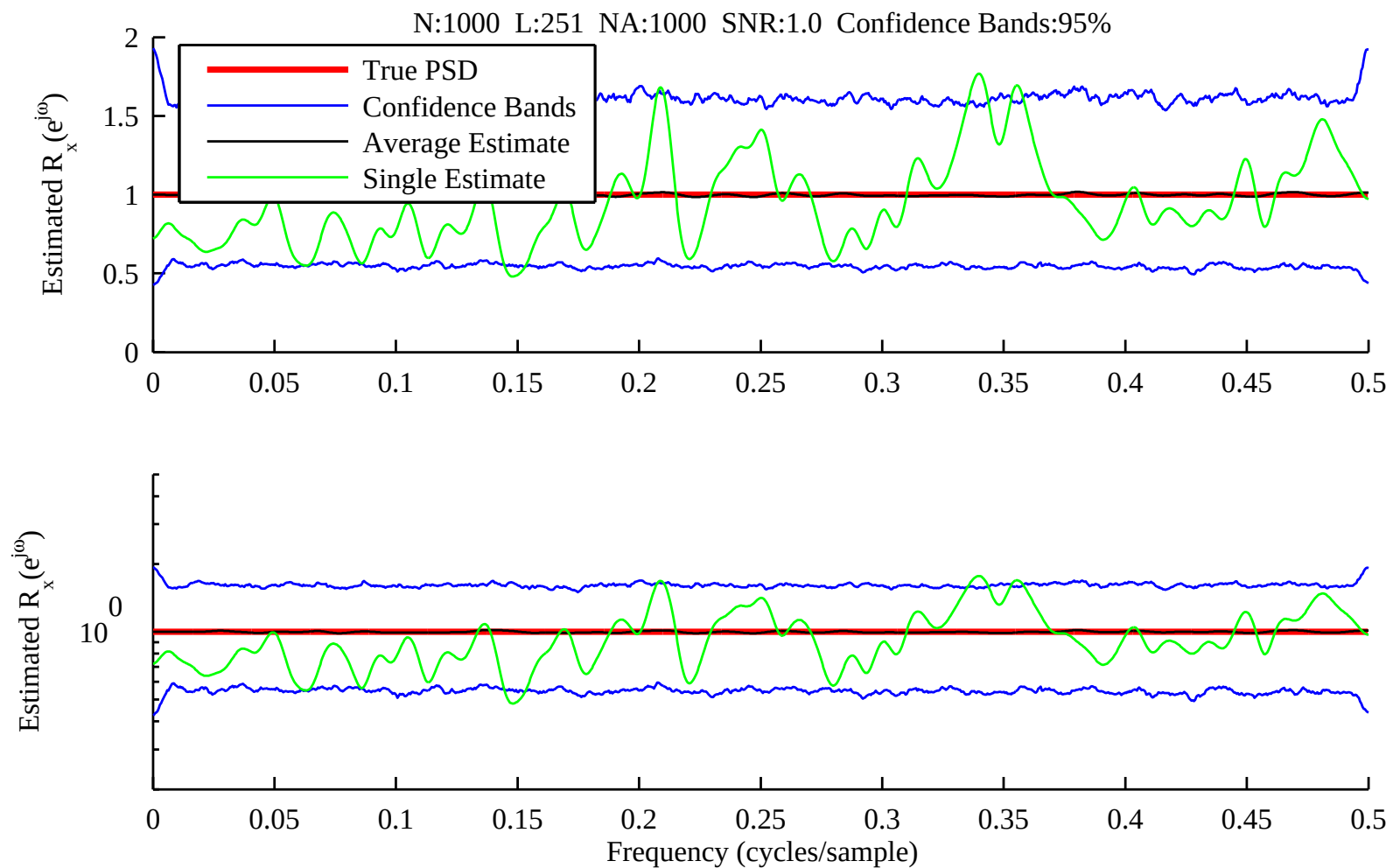
Example 2: Blackman-Tukey Estimate $|H(e^{j\omega})|$



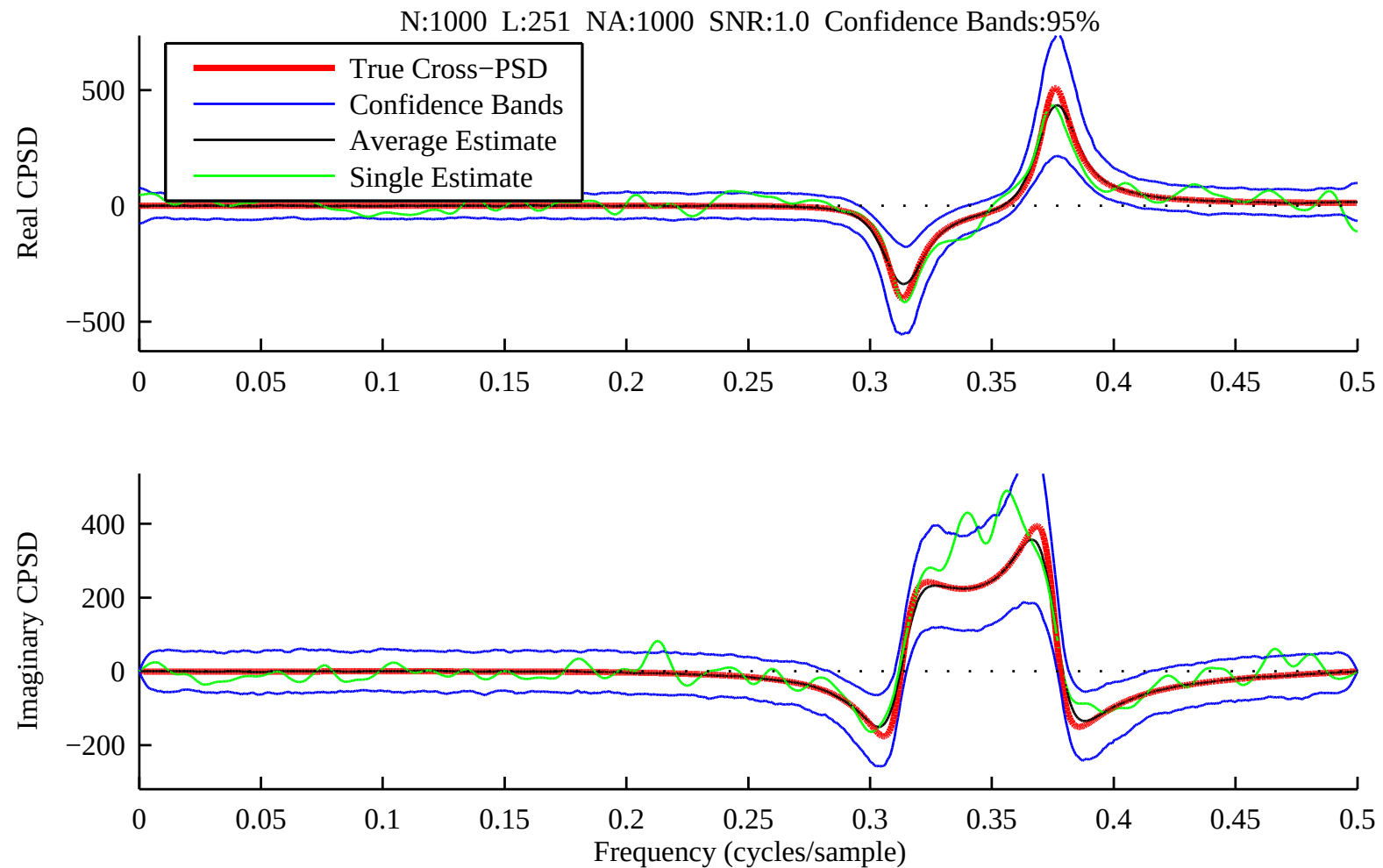
Example 2: Blackman-Tukey Estimate $\angle H(e^{j\omega})$



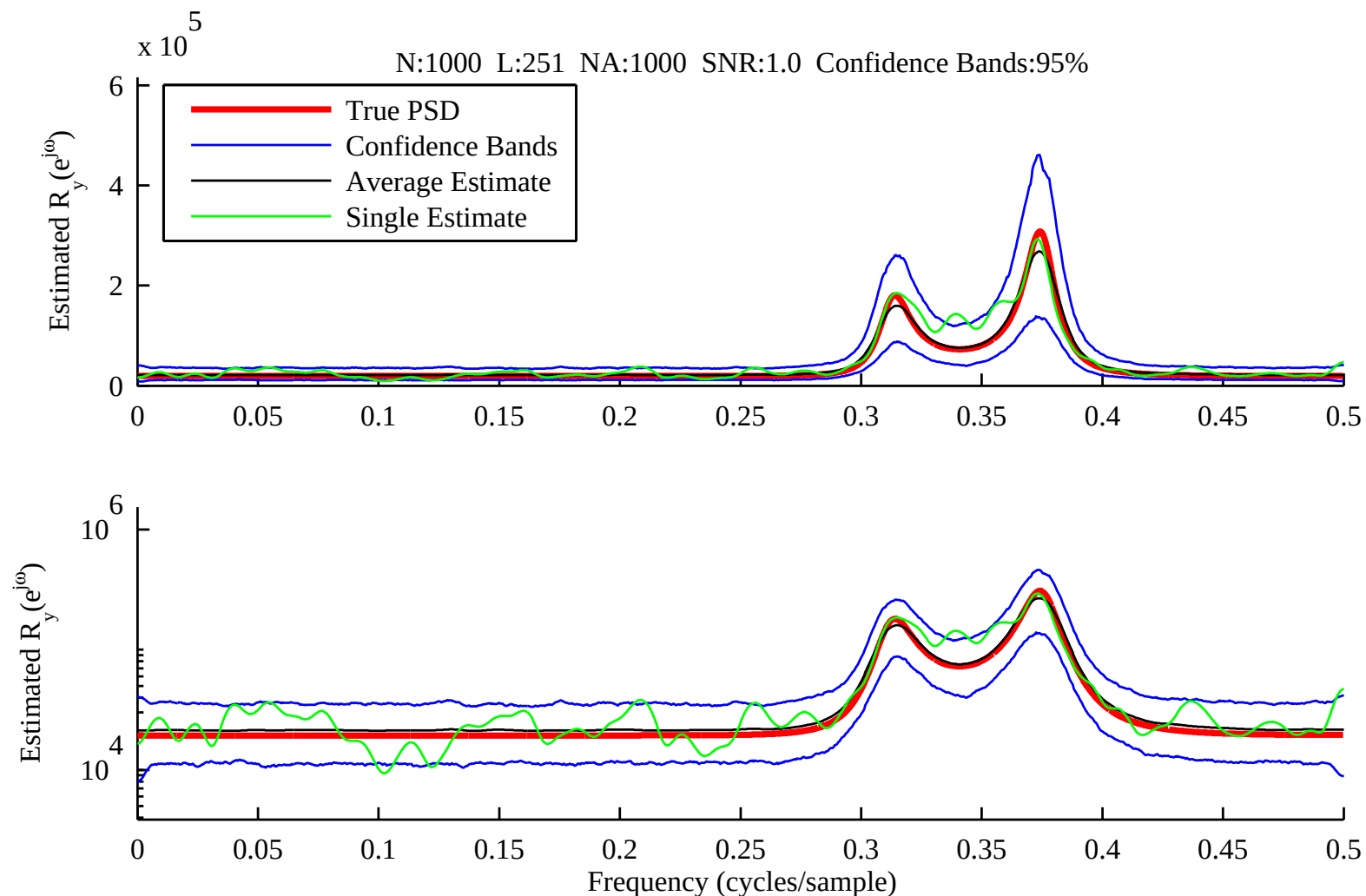
Example 2: Blackman-Tukey Estimate $R_x(e^{j\omega})$



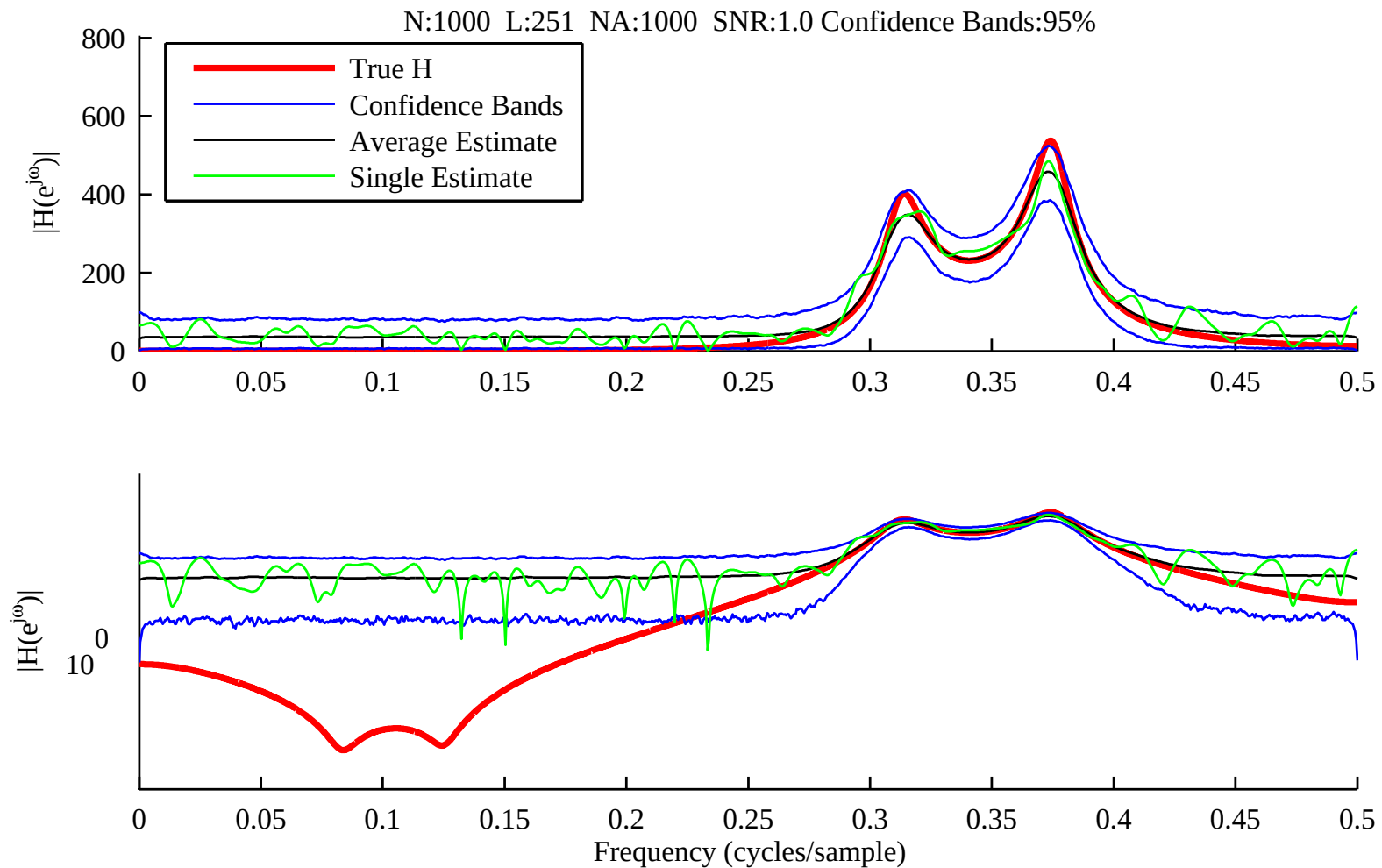
Example 2: Blackman-Tukey Estimate $R_{yx}(e^{j\omega})$



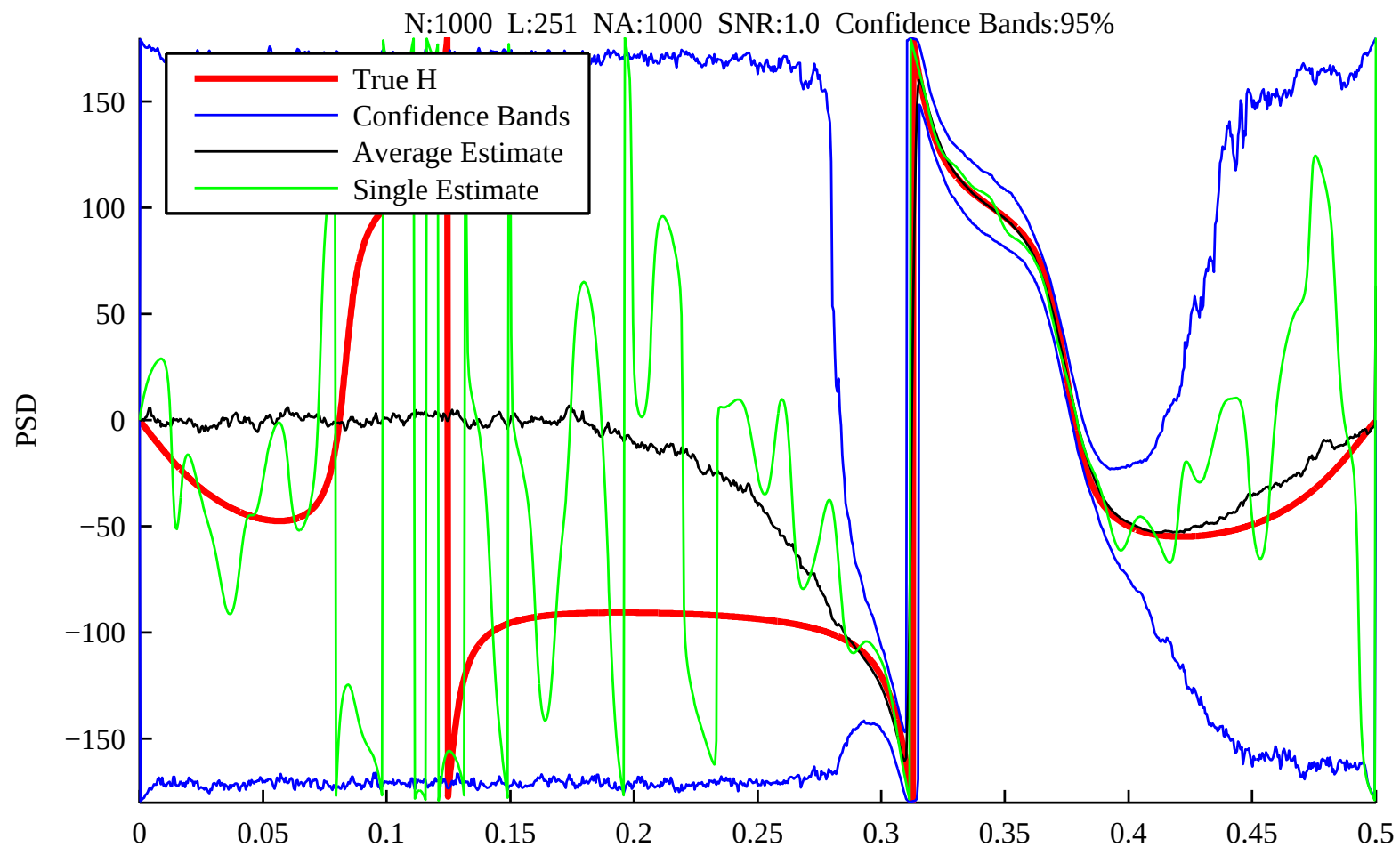
Example 2: Blackman-Tukey Estimate $R_y(e^{j\omega})$



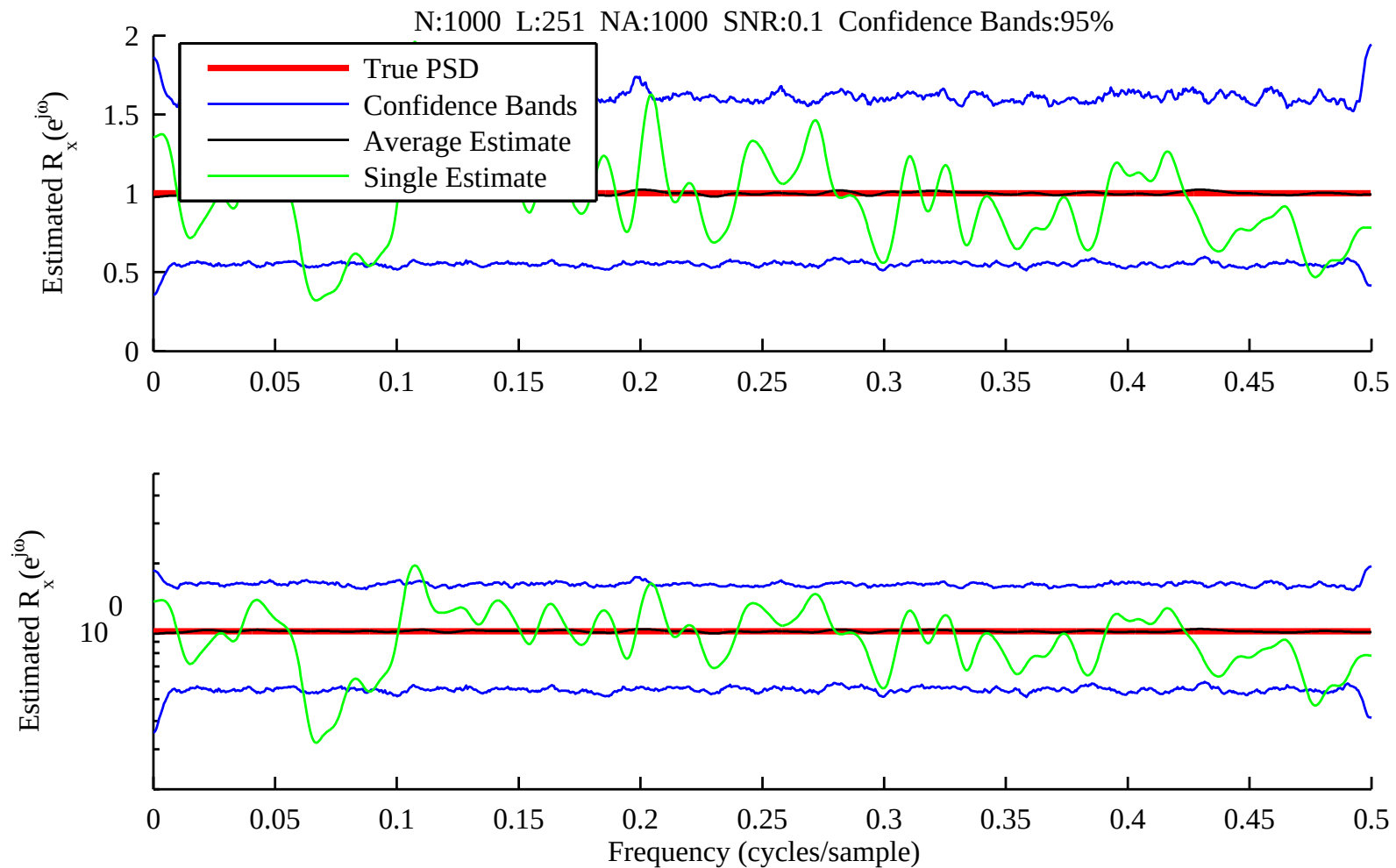
Example 2: Blackman-Tukey Estimate $|H(e^{j\omega})|$



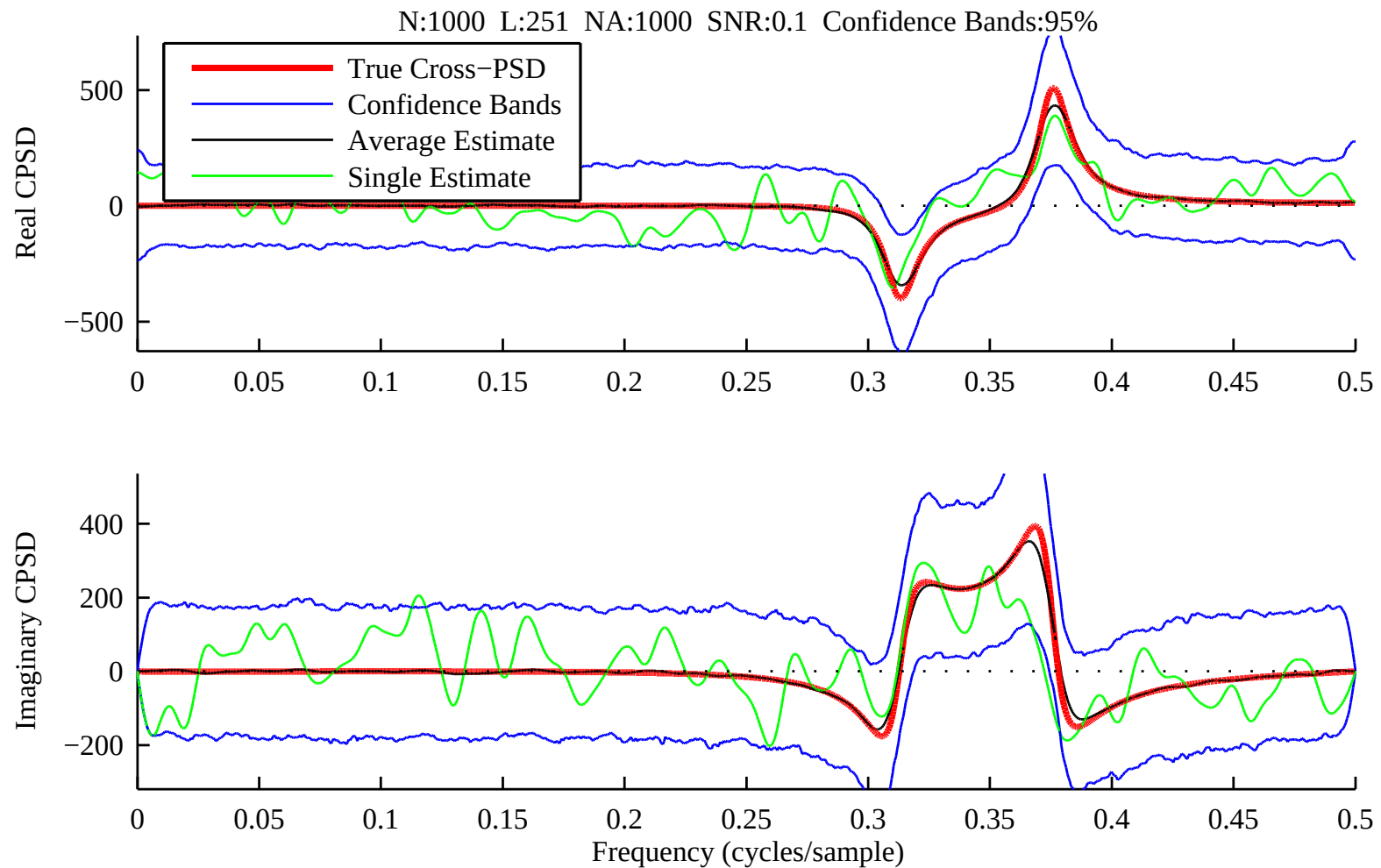
Example 2: Blackman-Tukey Estimate $\angle H(e^{j\omega})$



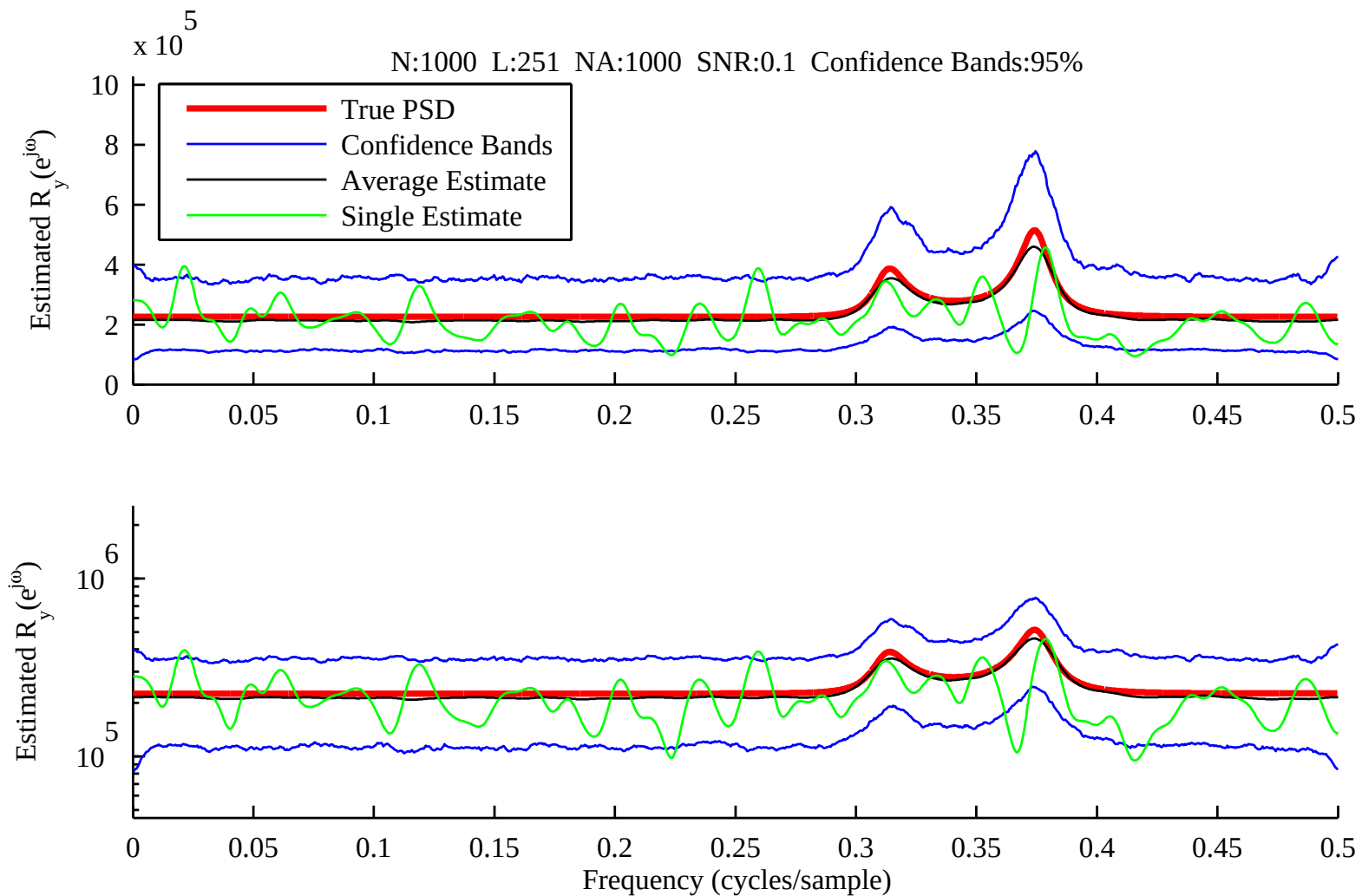
Example 2: Blackman-Tukey Estimate $R_x(e^{j\omega})$



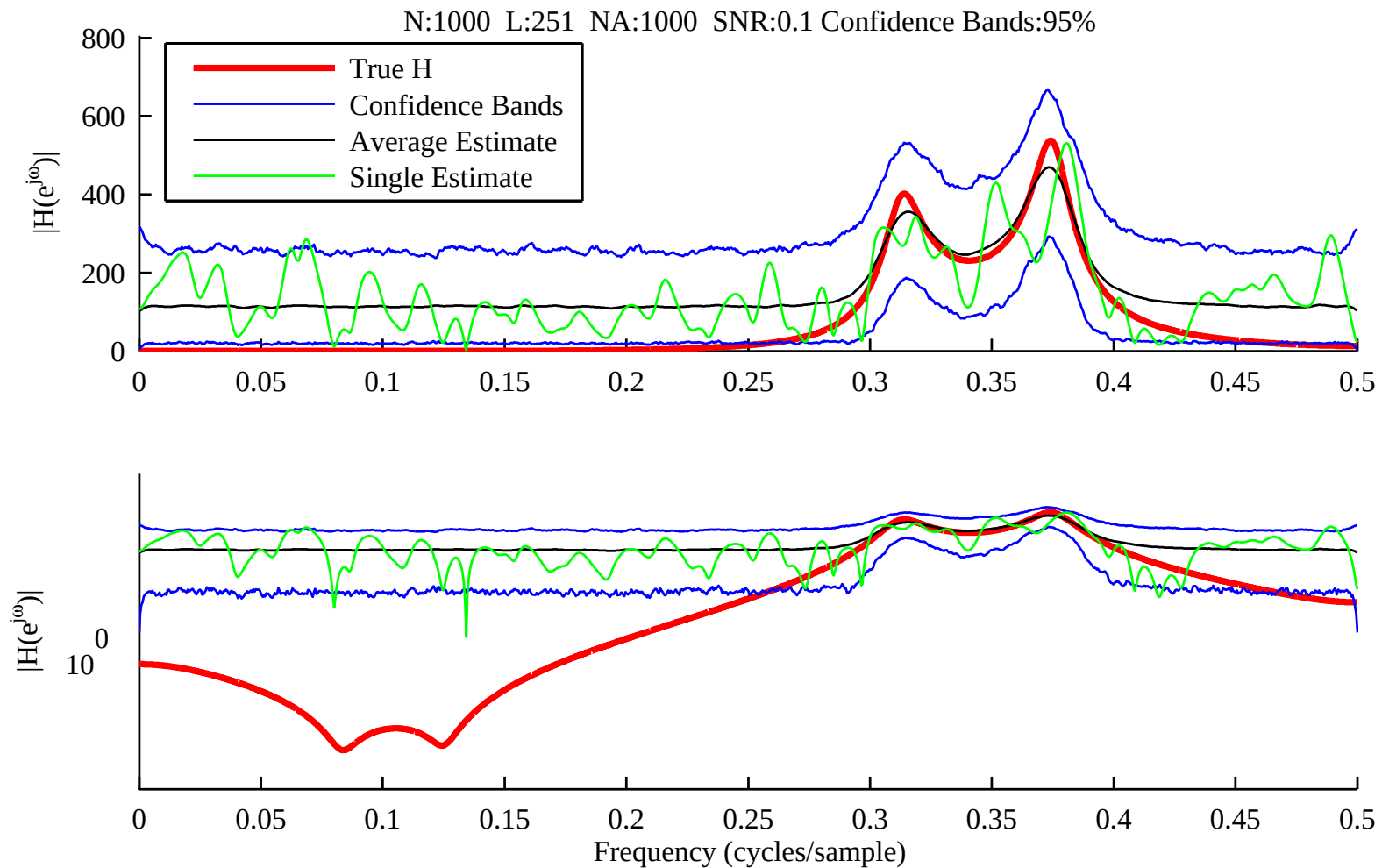
Example 2: Blackman-Tukey Estimate $R_{yx}(e^{j\omega})$



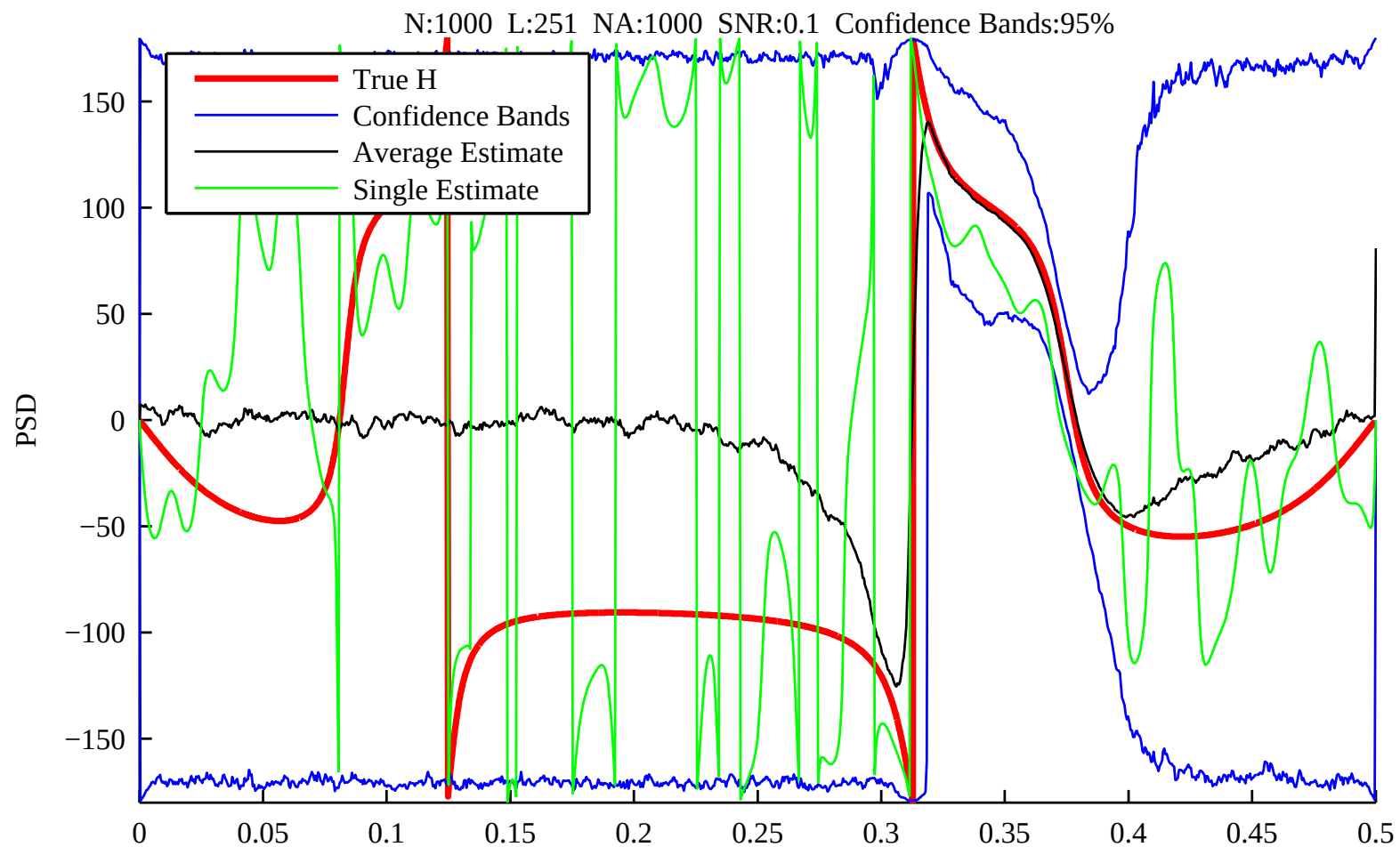
Example 2: Blackman-Tukey Estimate $R_y(e^{j\omega})$



Example 2: Blackman-Tukey Estimate $|H(e^{j\omega})|$



Example 2: Blackman-Tukey Estimate $\angle H(e^{j\omega})$



Example 2: MATLAB Code

```
%function [] = BlackmanTukeyEx();
close all;
clear all;

SNR = [10 1 0.1]; % MA filter order
L = 251;
N = 1000;          % Length of signal segment
NA = 1000;         % No. averages
P = 500;          % Padding to eliminate transient
cb = 95;           % Confidence Bands
np = 2^nextpow2(2*N-1); % Pad the signal to prevent overlap in the circular convolution
NZ = 2^11;         % No. points in the frequency domain from -pi to pi

a1 = 1-cb/100;     % Alpha

b = poly([-0.8,0.97*exp(j *pi/4),0.97*exp(-j *pi/4),...
          0.97*exp(j *pi/6),0.97*exp(-j *pi/6)]); % Numerator coefficients
a = poly([ 0.8,0.95*exp(j*3*pi/4),0.95*exp(-j*3*pi/4),...
          0.95*exp(j*2.5*pi/4),0.95*exp(-j*2.5*pi/4)]); % Denominator coefficients
b = b*sum(a)/sum(b); % Scale DC gain to 1

[R,w] = freqz(b,a,NZ);
H2 = abs(R).^2;
f = w/(2*pi);
Rx = 1*ones(size(R));
Ryx = R;
Hm = abs(R);
```

```

Ha      = angle(R)*180/pi;

%=====
% Plot Estimate
%=====
for c1 = 1:length(SNR),
    nx      = P+N;
    Rhx     = zeros(NA,NZ/2+1);
    Rhy     = zeros(NA,NZ/2+1);
    Rhyx    = zeros(NA,NZ/2+1);
    Hhm     = zeros(NA,NZ/2+1);
    Hha     = zeros(NA,NZ/2+1);
    kh      = 1:NZ/2+1;
    fh      = (kh-1)/NZ;

    no      = (L+1)/2;           % Number of one-sided points to included in estimate
    wn      = blackman(no*2-1);  % Create window
    wn      = wn/max(wn);        % Make maximum = 1
    v       = 2*N/(sum(wn.^2));

    for c2 = 1:NA,
        x    = randn(P+N,1);
        y    = filter(b,a,x); % System with known PSD
        npw  = var(y)/SNR(c1); % Noise power
        y    = y + randn(size(y))*sqrt(npw);
        y    = y(nx-N+1:nx); % Eliminate start-up transient (make stationary)
        x    = x(nx-N+1:nx); % Eliminate first set of samples

        xf   = fft(x,np);           % Calculate FFT of x
        rx   = ifft(xf.*conj(xf));  % Calculate biased autocorrelation
        rx   = real(rx(1:nx))/N;    % Eliminate superfluous zeros and nearly zero imaginary part
    end
end

```

```

    rx  = [rx(no:-1:2);rx(1:no)]; % Make two-sided, using symmetry
    rhx  = abs(fft(rx.*wn,NZ));    % Window autocorrelation and calculate the estimate

    yf  = fft(y,np);              % Calculate FFT of y
    ry  = ifft(yf.*conj(yf));      % Calculate biased autocorrelation
    ry  = real(ry(1:nx))/N;        % Eliminate superfluous zeros and nearly zero imaginary part
    ry  = [ry(no:-1:2);ry(1:no)]; % Make two-sided, using symmetry
    rhy  = abs(fft(ry.*wn,NZ));    % Window autocorrelation and calculate the estimate

    ryx  = ifft(yf.*conj(xf));      % Calculate biased cross-correlation
    ryx  = real([ryx(end-no+2:end);ryx(1:no)])/N;
    rhyx = exp(j*(0:NZ-1)'*(2*pi/NZ)*(no-1)).*fft(ryx.*wn,NZ); % Window crosscorrelation and calculate

    Rhx (c2,:) = rhx(kh).';
    Rhy (c2,:) = rhy(kh).';
    Rhyx(c2,:) = rhyx(kh).';
    Hhm (c2,:) = abs(Rhyx(c2,:)./Rhx(c2,:));
    Hha (c2,:) = rem(angle(Rhyx(c2,:))*180/pi,180);
end;

%=====
% Plot Rhx & Rhy
%=====
for c2=1:2,
    switch c2,
        case 1,
            R  = Rx;
            Rha = mean(Rhx); % Average
            Rhu = prctile(Rhx,100-(100-cb)/2); % Upper confidence band
            Rhl = prctile(Rhx,(100-cb)/2); % Lower confidence band
            Rh1 = Rhx(1,:);

```

```

        fn    = sprintf('BTRx%03d',SNR(c1)*10);
        yl    = 'Estimated R_x(e^{j\omega})';
    case 2,
        R      = H2 + npw;
        Rha    = mean(Rhy); % Average
        Rhu    = prctile(Rhy,100-(100-cb)/2); % Upper confidence band
        Rhl    = prctile(Rhy,(100-cb)/2); % Lower confidence band
        Rh1    = Rhy(1,:);
        fn     = sprintf('BTRy%03d',SNR(c1)*10);
        yl     = 'Estimated R_y(e^{j\omega})';
    end;
figure;
FigureSet(1,'LTX');
subplot(2,1,1);
    h = plot(f,R,'r',fh,Rhl,'b',fh,Rhu,'b',fh,Rha,'k',fh,Rh1,'g');
    set(h(1), 'LineWidth',1.3);
    set(h(2:3), 'LineWidth',0.5);
    set(h(4), 'LineWidth',0.5);
    set(h(5), 'LineWidth',0.5);
    box off;
    ylabel(yl);
    title(sprintf('N:%d L:%d NA:%d SNR:%3.1f Confidence Bands:%d%%',N,L,NA,SNR(c1),cb));
    xlim([0 0.5]);
    ylim([0 max(R)*2.]);
    AxisSet(8);
    legend(h([1 3 4 5]),'True PSD','Confidence Bands','Average Estimate','Single Estimate',2);
subplot(2,1,2);
    h = semilogy(f,R,'r',fh,Rhl,'b',fh,Rhu,'b',fh,Rha,'k',fh,Rh1,'g');
    set(h(1), 'LineWidth',1.3);
    set(h(2:3), 'LineWidth',0.5);
    set(h(4), 'LineWidth',0.5);

```

```

        set(h(5) , 'LineWidth', 0.5);
        box off;
        ylabel(y1);
        xlabel('Frequency (cycles/sample)');
        xlim([0 0.5]);
        ylim([0.2*min(R) max(R)*5]);
    AxisSet(6);
    print(fn, '-depsc');
end;

%=====
% Plot Rhyx
%=====
Rha = mean(Rhyx); % Average
Rhur = prctile(real(Rhyx), 100-(100-cb)/2); % Upper confidence band
Rhlr = prctile(real(Rhyx), (100-cb)/2); % Lower confidence band
Rhui = prctile(imag(Rhyx), 100-(100-cb)/2); % Upper confidence band
Rhli = prctile(imag(Rhyx), (100-cb)/2); % Lower confidence band

figure;
FigureSet(2, 'LTX');
subplot(2,1,1);
h = plot(f, real(Ryx), 'r', fh, Rhlr, 'b', fh, Rhur, 'b', fh, real(Rha), 'k', fh, real(Rhyx(1,:)), 'g');
set(h(1) , 'LineWidth', 1.3);
set(h(2:3), 'LineWidth', 0.5);
set(h(4) , 'LineWidth', 0.5);
set(h(5) , 'LineWidth', 0.5);
box off;
ylabel('Real CPSD');
title(sprintf('N:%d L:%d NA:%d SNR:%3.1f Confidence Bands:%d%%', N, L, NA, SNR(c1), cb));

```

```

        xlim([0 0.5]);
        ymn = min(real(Ryx));
        ymx = max(real(Ryx));
        yrg = ymx-ymn;
        ylim([ymn-0.25*yrg ymx+0.25*yrg]);
        AxisLines;
        AxisSet(8);
        legend(h([1 3 4 5]), 'True Cross-PSD', 'Confidence Bands', 'Average Estimate', 'Single Estimate' 2);
subplot(2,1,2);
h = plot(f, imag(Ryx), 'r', fh, Rhli, 'b', fh, Rhui, 'b', fh, imag(Rha), 'k', fh, imag(Rhyx(1,:)), 'g');
set(h(1), 'LineWidth', 1.3);
set(h(2:3), 'LineWidth', 0.5);
set(h(4), 'LineWidth', 0.5);
set(h(5), 'LineWidth', 0.5);
box off;
ylabel('Imaginary CPSD');
xlabel('Frequency (cycles/sample)');
ymn = min(imag(Ryx));
ymx = max(imag(Ryx));
yrg = ymx-ymn;
ylim([ymn-0.25*yrg ymx+0.25*yrg]);
AxisLines;
AxisSet(6);
print(sprintf('BTRyx%03d', SNR(c1)*10), '-depsc');

%=====
% Plot Hh Magnitude
%=====
Hhav = mean(Hhm); % Average
Hhu = prctile(Hhm, 100-(100-cb)/2); % Upper confidence band
Hhl = prctile(Hhm, (100-cb)/2); % Lower confidence band

```

```

figure;
FigureSet(3,'LTX');
subplot(2,1,1);
    h = plot(f,Hm,'r',fh,Hhl,'b',fh,Hhu,'b',fh,Hhav,'k',fh,Hhm(1,:), 'g');
    set(h(1)    , 'LineWidth',1.3);
    set(h(2:3)  , 'LineWidth',0.5);
    set(h(4)    , 'LineWidth',0.5);
    set(h(5)    , 'LineWidth',0.5);
    box off;
    ylabel(' |H(e^{j\omega})| ');
    title(sprintf('N:%d  L:%d  NA:%d  SNR:%3.1f  Confidence Bands:%d%%',N,L,NA,SNR(c1),cb));
    xlim([0 0.5]);
    ylim([0 max(Hm)*1.5]);
    AxisSet(8);
    legend(h([1 3 4 5]), 'True H', 'Confidence Bands', 'Average Estimate', 'Single Estimate',2);
subplot(2,1,2);
    h = semilogy(f,Hm,'r',fh,Hhl,'b',fh,Hhu,'b',fh,Hhav,'k',fh,Hhm(1,:), 'g');
    set(h(1)    , 'LineWidth',1.3);
    set(h(2:3)  , 'LineWidth',0.5);
    set(h(4)    , 'LineWidth',0.5);
    set(h(5)    , 'LineWidth',0.5);
    box off;
    ylabel(' |H(e^{j\omega})| ');
    xlabel('Frequency (cycles/sample)');
    xlim([0 0.5]);
    ylim([0.2*min(Hm) max(Hm)*5]);
AxisSet(6);
print(sprintf('BTH%03d',SNR(c1)*10), '-depsc');

%fprintf('Pausing...\n'); pause;

```

```

%continue;
%=====
% Plot Hh Phase
%=====
Hhav = mean(Hha); % Average
Hhu = prctile(Hha,100-(100-cb)/2); % Upper confidence band
Hhl = prctile(Hha, (100-cb)/2); % Lower confidence band

figure;
FigureSet(4,'LTX');
h = plot(f,Ha,'r',fh,Hhl,'b',fh,Hhu,'b',fh,Hhav,'k',fh,Hha(1,:), 'g');
set(h(1) , 'LineWidth',1.3);
set(h(2:3), 'LineWidth',0.5);
set(h(4) , 'LineWidth',0.5);
set(h(5) , 'LineWidth',0.5);
box off;
ylabel('PSD');
title(sprintf('N:%d L:%d NA:%d SNR:%3.1f Confidence Bands:%d%%',N,L,NA,SNR(c1),cb));
xlim([0 0.5]);
ylim([-180 180]);
AxisSet(6);
legend(h([1 3 4 5]),'True H','Confidence Bands','Average Estimate','Single Estimate',2);
print(sprintf('BTP%03d',SNR(c1)*10),'-depsc');

%fprintf('Pausing...\n'); pause;
end;

```

Example 2: Observations

- Phase can be estimated accurately at frequencies where SNR is high

Summary

- If we have two signals for a stochastic process, we can estimate much more than with just a single signal
 - Cross-correlation, Cross-covariance
 - Cross-power spectral density
 - Transfer function estimation
- Can use same techniques for the single signal case
- Will discuss coherence in a separate set of slides